Centro Universitário de Maringá
Curso: ENGENHARIA DE SOFTWARE

JULIA OLIVEIRA DOS SANTOS **RA 25000238-2**
JULIA YUMI HASHIMOTO **RA 250001309-2**

OS PRINCIPAIS MÉTODOS DE CLASSIFICAÇÃO E ORDENAÇÃO EM LINGUAGEM C.

MARINGÁ
2025

JULIA OLIVEIRA DOS SANTOS **RA 25000238-2**

JULIA YUMI HASHIMOTO **RA 250001309-2**

Metodos de Classificação e Ordenação - YouTube

OS PRINCIPAIS MÉTODOS DE CLASSIFICAÇÃO E ORDENAÇÃO EM LINGUAGEM C.

Trabalho de pesquisa apresentado à disciplina de Linguagem e Técnicas de Programação, como requisito para obtenção de nota avaliativa, sob orientação do Prof. Dr. João Marcos Ferlini.

MARINGÁ

2025

SUMÁRIO

1. RESUMO EXECUTIVO 3
2. INTRODUÇÃO 4
3. MÉTODOS DE CLASSIFICAÇÃO E ORDENAÇÃO 5
4. DESCRIÇÃO DOS PRINCIPAIS MÉTODOS E EXEMPLOS DE CÓDIGO 5
4.1. Bubble Sort 5
4.2. Selection Sort 6
4.3. Insertion Sort 7
4.4. Merge Sort 8
4.5. Quick Sort 9
5. ANÁLISE E COMPARAÇÃO 10
5.1. Memória 10
5.2. Vantagens e Desvantagens 10
5.3. Complexidade 11
5.4. Cenário de Uso 12
6. CONCLUSÃO 13
REFERÊNCIAS 13

1. RESUMO EXECUTIVO

A crescente necessidade de organizar grandes volumes de dados em sistemas computacionais exige métodos eficientes de ordenação, fundamentais para a otimização de processos e a redução de tempo de processamento. Este trabalho apresenta uma análise comparativa dos principais algoritmos de ordenação implementados na linguagem C, incluindo **Bubble Sort, Insertion Sort, Selection Sort, Merge Sort e Quick Sort**. O objetivo central é avaliar o desempenho, a eficiência e a aplicabilidade de cada algoritmo em diferentes cenários, considerando fatores como tempo de execução, uso de memória e complexidade computacional.

Para isso, são realizadas implementações práticas em C, permitindo observar o funcionamento real de cada método e comparar seus resultados em conjuntos de dados variados. A análise inclui também referências a estudos acadêmicos, consolidando informações sobre eficiência, vantagens e limitações de cada técnica. O estudo busca fornecer uma visão completa, evidenciando a importância da escolha adequada do algoritmo para otimizar o processamento de dados, melhorar a organização de sistemas e aprimorar o desempenho de projetos de software.

Palavras-chave: Ordenação. Complexidade. Desempenho. Eficiência. Procedimentos. Constante. Comparação

2. INTRODUÇÃO

Ordenar informações de maneira adequada permite otimizar operações como pesquisa, análise e manipulação de dados, tornando os processos mais ágeis e confiáveis. Diferentes algoritmos de ordenação apresentam características variadas, sendo mais ou menos eficientes dependendo do tamanho e da estrutura dos conjuntos de dados. Neste contexto, este trabalho analisará e comparará os principais métodos de ordenação implementados na linguagem C, **incluindo Bubble Sort, Insertion Sort, Selection Sort, Merge Sort e Quick Sort**. A abordagem combina explicações teóricas com implementações práticas, permitindo observar o comportamento de cada algoritmo em diferentes cenários. Serão avaliados aspectos como tempo de execução, complexidade computacional e uso de memória, oferecendo uma visão detalhada das vantagens, limitações e aplicações de cada técnica.

3. MÉTODOS DE CLASSIFICAÇÃO E ORDENAÇÃO

Algoritmos de ordenação são procedimentos que organizam ou reorganizam os valores de uma sequência, permitindo que os dados sejam acessados e utilizados de forma mais eficiente. Um dos usos mais comuns desses algoritmos é a ordenação de vetores, onde cada variável pode armazenar diversas posições de acordo com o tamanho definido. Por exemplo, uma lista de produtos em um supermercado pode ser organizada por preço crescente, facilitando a comparação e a escolha dos itens pelos clientes.

4. DESCRIÇÃO DOS PRINCIPAIS MÉTODOS E EXEMPLOS DE CÓDIGO

4.1. Bubble Sort

O Bubble Sort é um dos algoritmos mais simples e funciona comparando dois elementos de um vetor e trocando-os de posição quando necessário. Conforme descrito por Pereira (2010, p.95), “Após a primeira passagem completa pelo vetor (...) podemos garantir que o maior item terá sido deslocado para a última posição do vetor”. Esse processo se repete até que todo o vetor esteja completamente ordenado.

```c
#include <stdio.h>
#include <locale.h>
//Algoritmo de ordenação - Bubble Sort
//O nome bubble vem da referência das bolhas de ar que sobem com a respiração na água.

int main (){
    setlocale (LC_ALL,"Portuguese");

    int vetor[5], i, auxiliar;
    //Recomendado para conjuntos pequenos.

    int troca=1;
    //inicializa a troca dos números como sendo verdadeira

    printf("Digite 5 números inteiros:\n ");
    for (i=0; i<5; i++){
        printf ("Número %d: ", i+1);
        scanf ("%d", &vetor[i]);
    }

    //O uso do "while" otimiza o percurso do código, caso o
    //vetor já estiver na ordem desejada.

    //ao invés de utilizar o "for", que iria percorrer o
    //vetor completamente, mesmo ele estando certo.
    while(troca){
        troca=0;//confere se vai haver alguma troca dentro do for.
        for(i=0;i<4;i++) {
            if (vetor[i]>vetor[i+1]){ //Condição para a ordem crescente.
                auxiliar=vetor[i]; //processo de troca dos elementos.
                vetor[i]=vetor[i+1];
                vetor[i+1]=auxiliar; //utilizamos a variável auxiliar para guardar o valor
                //original de "vetor[i]" tmporarioamente.
                troca=1;//troca recebe verdadeiro caso o if for verdade.
                //se troca receber 0, o while não irá repetir.

            }
        }
    }

    printf ("Resultado do vetor em ordem crescente: \n");
    for (i=0;i<5;i++){
        printf ("%d\n", vetor[i]);
    }

}
```

4.2. Selection Sort

O Selection Sort também é um algoritmo simples e bastante usado. Ele organiza o vetor escolhendo o menor elemento e colocando-o na primeira posição, depois pega o próximo menor e coloca na segunda posição, repetindo esse processo até que todos os elementos estejam ordenados. No final, cada valor à esquerda

sempre será menor do que os valores que estão à sua direita.

```c
#include <stdio.h>
#include <locale.h>
//Algoritmo de ordenação - Selection Sort

int main (){
    setlocale (LC_ALL,"Portuguese");

    int vetor[5], i, j, menor, auxiliar;

    printf ("informe 5 valores para o vetor\n ");

    for (i=0; i<5; i++) {
        printf ("Informe o %dº número: ",i+1);
        scanf ("%d", &vetor[i]);
    }

    for (i=0; i<5; i++) { //repetição externa, para controlar o inicio e o fim do vetor.
        menor= i; //inicilialização para a troca do menor número.

        for (j=i+1; j<5; j++) { //repetição interna, usada para fazer a comparação.
            if(vetor[j]< vetor[menor]) //condição para a troca.
            menor= j; //caso o if for verdadeiro, o "menor" recebe o indicie do número encontrado.
    }
            auxiliar = vetor[i]; //processo de troca
            vetor[i] = vetor[menor];
            vetor[menor] = auxiliar;

        }
        printf ("Resultado da ordem crescente dos números:\n ");
        for (i=0;i<5;i++) {
            printf ("%d\n", vetor[i]);
        }
    }
```

4.3. Insertion Sort

Insertion Sort é muito semelhante ao Bubble Sort É um algoritmo de ordenação simples e de fácil de implementação (SZWARCFTIRE & MARKENZON, 2015). Ele funciona comparando os elementos e inserindo cada um na posição correta dentro do vetor. À medida que percorre o vetor, o algoritmo organiza os elementos um a um, de forma que cada elemento fique com valores menores à sua **esquerda** e valores

maiores à sua **direita.**

```c
#include <stdio.h>
#include <locale.h>
//Algoritmo de ordenação - Insertion Sort
//Lógica baseada na inserção ordenada.

int main (){
    setlocale (LC_ALL,"Portuguese");

    int vetor[5], i, j, auxiliar;

    printf ("informe 5 valores para o vetor\n ");

    for (i=0; i<5; i++) {
        printf ("Informe o %dº número: ",i+1);
        scanf ("%d", &vetor[i]);
    }

    for (i=1;i<5;i++) { //O "i" começa no índice 1 pois
    //começa fazendo comparação com o número do índice 0.

        j=i; //Inicialização do váriavel j.
        auxiliar = vetor[j]; //armazenamento do primeiro número a ser comparado.

        while (j>0 && auxiliar < vetor[j-1]) { //verificação da posição
        //do indice e condição para a troca.
            vetor[j] = vetor [ j-1]; //processo de troca do elemento para tràs
            j--; //índice volta uma posição para a esquerda, evitando um loop
            //e garantindo a comparação.
        }

        vetor[j] = auxiliar;
    }
        printf ("Resultado da ordem crescente dos números:\n ");
        for (i=0;i<5;i++) {
            printf ("%d\n", vetor[i]);
        }
}
```

4.4. Merge Sort

O Merge Sort é um algoritmo de ordenação que organiza um vetor dividindo-o em partes menores e depois juntando essas partes de forma ordenada. Ele funciona em etapas: primeiro, o vetor é quebrado em grupos menores até que cada grupo tenha apenas um número. Por exemplo, se o vetor tem 8 números, ele é dividido em dois grupos de 4, depois cada grupo de 4 é dividido em dois grupos de 2, e assim por diante, até que cada grupo tenha só um elemento.

Em seguida, os grupos são juntados aos poucos, comparando os números e colocando-os em ordem crescente. Dois números são comparados e organizados, formando pequenos grupos ordenados. Depois, esses grupos já organizados são combinados com outros grupos, mantendo a ordem correta. Esse processo se repete até que todos os números do vetor estejam reunidos em um único vetor totalmente ordenado.

O Merge Sort funciona dessa maneira porque é mais fácil organizar números em grupos pequenos. Ao unir essas partes já ordenadas, garantimos que o vetor também fique organizado de forma eficiente.

```c
1   #include <stdio.h>
2   #include <locale.h>
3   #include <stdlib.h> //biblioteca necessária para a utilizar o "malloc"
4   //que irá criar um espaço na memória quanto o programa está rodando.
5
6   //Algoritmo de ordenação - Merge Sort (utilizando método recursivo)
7   //Divide o vetor em duas metades, ordena recursivamente essas metades
8   //e depois intercala as partes ordenadas em um novo vetor.
9   //Alta eficiência e aguenta grandes volumes de informação.
10
11  void merge (int vetor[], int inicio, int meio, int fim) {
12  //função que intercala as duas subpartes ordenadas do vetor.
13      int i, j, k, n1=meio - inicio + 1;
14      //cálculo para a quantidade de índices do vetor esquerdo.
15      int n2 = fim - meio;
16      //cálculo de quant de índices para o vetor direito.
17
18      int *esquerda = malloc(n1 *sizeof(int));
19      int *direita = malloc(n2 *sizeof(int));
20      //linhas 18 e 19 utilizadas para fazer uma locação dinâmica de memória
21      //necessário para conjuntos grandes.
22
23      for (i=0; i<n1; i++)
24          esquerda[i] = vetor[inicio+i];
25
26      for (j=0;j<n2;j++)
27          direita[j] = vetor[meio+1+j];
28      //linhas 23 até 27 utilizadas para copiar os índices do vetor original para os subvetores
29
30      i=0; j=0; k=inicio; //processo de merge, onde a organização dos números será feita e comparada.
31      while (i<n1 && j<n2){
32          if (esquerda[i]<= direita[j])
33              vetor[k++]= esquerda[i++];
34              //copia do número da esquerda (para o vetor com índice k) de acordo com a condição e incremento do índice k (vetor principal).
35          else
36              vetor[k++]= direita[j++];
37              //copia do número da direita (para o vetor com índice k) de acordo com a condição e incremento do índice k (vetor principal).
38      }
39
40      while (i<n1)
41          vetor[k++] = esquerda[i++];
42      while (j<n2)
43          vetor[k++]=direita[j++];
44      //linhas 40 até 43 fazem o processo de copia dos elementos restantes e verificação.
45
46      free(esquerda);
47      free(direita);
48      //liberação de memória dos ponteiros.
49  }
50
51  void mergeSort(int vetor[], int inicio, int fim) { //função principal que realiza as divisões com chamada
52  //recursiva.
53      if (inicio<fim) {
54          int meio= inicio + (fim - inicio) /2;
55          mergeSort(vetor, inicio, meio);
56          mergeSort(vetor, meio+1, fim);
57          merge(vetor, inicio, meio, fim);
58      }
59  }
60
61  int main (){
62      setlocale (LC_ALL,"Portuguese");
63
64          int tam=5,vetor[tam], i;
65
66      printf ("informe 5 valores para o vetor\n ");
67
68      for (i=0; i<5; i++) {
69          printf ("Informe o %dº número: ",i+1);
70          scanf ("%d", &vetor[i]);
71      }
72
73      mergeSort(vetor,0,tam-1);
74
75      printf ("Resultado do vetor em ordem crescente: \n");
76      for (i=0;i<tam;i++){
77          printf ("%d\n", vetor[i]);
78
79      }
80  }
```

4.5. Quick Sort

O Quick Sort funciona dividindo o vetor em partes menores, mas de forma diferente do Merge Sort. Ele escolhe um elemento do vetor chamado pivô. Em seguida, organiza os números em duas partes: à esquerda ficam os elementos menores que o pivô e à direita os elementos maiores que o pivô.

Depois, o mesmo procedimento é repetido em cada uma dessas partes, escolhendo novos pivôs e dividindo novamente, até que todos os elementos estejam organizados em ordem crescente (SZWARCFITER & MARKENZON, 2015).

```c
1  #include <stdio.h>
2  #include <locale.h>
3  #include <stdlib.h>
4  //Algoritmo de ordenação - Quick Sort
5
6  int divisao(int vetor[5], int inicio, int fim){ //função que retorna o índice do elemento do meio, já posicionado corretamente.
7      int ref =( vetor[inicio]+vetor[fim]+vetor[(inicio+fim)/2])/3;
8      //cálculo para o número de referência para comparação.
9
10     while (inicio<fim) {
11         while(inicio<fim && vetor[inicio]<= ref)
12          inicio++;
13     //linhas 11 e 12 utilizadas para percorrer o vetor da esquerda para a direita
14     //até encontrar um elemento maior que a ref.
15         while(inicio<fim && vetor[fim]> ref)
16             fim = fim - 1;
17     //linhas 15 e 16 utilizadas para percorrer o vetor da direita  para a esquerda
18     //até encontrar um elemento menor que a ref.
19         int auxiliar = vetor[inicio];
20         vetor[inicio]= vetor[fim];
21         vetor[fim]=auxiliar;
22     //linhas 19 e 21 utilizadas para fazer a troca dois dois números.
23     }
24     return inicio;
25     //Neste momento o vetor esta divido entre números menores e maiores que o número de encontro entre o i e f
26     //necessita-se fazer a organização desses números.
27 }
```

```c
28 void quickSort(int vetor[5], int inicio, int fim) { //quickSort vai executar sempre
29 //que o inicio for menor que o fim.
30     if(inicio<fim)  { //condição de parada.
31       int posicao=divisao(vetor,inicio,fim);
32       quickSort(vetor, inicio, posicao-1);
33       quickSort(vetor, posicao,fim);
34     }
35 }
36     int main (){
37     setlocale(LC_ALL,"Portuguese");
38
39     int vetor[5], i;
40
41     printf ("informe 5 valores para o vetor\n ");
42
43     for (i=0; i<5; i++) {
44         printf ("Informe o %dº número: ",i+1);
45         scanf ("%d", &vetor[i]);
46     }
47     quickSort(vetor,0,5-1);
48
49         printf ("Resultado da ordem crescente dos números:\n ");
50         for (i=0;i<5;i++) {
51             printf ("%d\n", vetor[i]);
52         }
53 }
54
```

5. ANÁLISE E COMPARAÇÃO

5.1. Memória

- **Bubble Sort:** utiliza memória extra constante (O (1)), pois ordena in-place.
- **Selection Sort:** também requer apenas memória adicional constante (O (1)), sendo in-place.
- **Insertion Sort:** ainda in-place, apenas variáveis auxiliares, logo O (1) de memória extra.
- **Merge Sort:** requer memória auxiliar proporcional ao tamanho da entrada (O(n)), para realizar o processo de intercalação (“merge”).
- **Quick Sort:** utiliza pouco espaço extra, tipicamente o espaço para a pilha de recursão, em média O (log n).

5.2. Vantagens e Desvantagens

- **Bubble Sort:**

  Vantagens: implementação muito simples; ótima para ensino, demonstrações e vetores muito pequenos.

  Desvantagens: é extremamente ineficiente para dados maiores; número elevado de comparações e trocas;

- **Selection Sort:**

  Vantagens: número reduzido de trocas (o que pode ser útil quando trocar elementos é caro); implementação simples.

  Desvantagens: tempo quadrático em qualquer situação de uso ($O(n^2)$); não se adapta a dados quase ordenados; não estável.

- **Insertion Sort**

  Vantagens: excelente desempenho quando os dados estão quase ordenados; simples de implementar; algoritmo estável.

  Desvantagens: para grandes volumes de dados e desordenados, ainda será lento ($O(n^2)$).

- **Merge Sort**

  Vantagens: desempenho garantido de O (n log n) em todos os casos; algoritmo estável; muito adequado para grandes volumes de dados.

  Desvantagens: exige memória extra (vetor auxiliar); maior complexidade de implementação; pode ter overflow em casos de vetores muito pequenos.

- **Quick Sort**

  Vantagens: um dos algoritmos mais rápidos na prática para muitos conjuntos de dados; pequeno uso de memória extra; bom equilíbrio geral.

  Desvantagens: pior caso pode ser $O(n^2)$ (dependendo da escolha de pivô); não é estável; em situações adversas podem degradar.

5.3. Complexidade

- **Bubble Sort:** melhor caso ~ O(n) (com otimização e já ordenado), caso médio e pior caso ~ $O(n^2)$.

- **Selection Sort:** constante ~ $O(n^2)$ para melhor, médio e pior caso — não se beneficia de pré-ordenamento.
- **Insertion Sort:** melhor ~ $O(n)$ quando quase ordenado; médio e pior ~ $O(n^2)$.
- **Merge Sort:** melhor, médio e pior todos ~ $O(n \log n)$.
- **Quick Sort:** melhor e médio ~ $O(n \log n)$; pior ~ $O(n^2)$ se o pivô for mal escolhido ou os dados estiverem em configuração adversa.

5.4. Cenário de Uso

- **Bubble Sort:** Adequado para ensino, demonstração ou vetores muito pequenos onde desempenho não é crítico.
- **Selection Sort:** Útil quando poucas trocas são desejadas e memória extra é muito limitada — ainda em vetores reduzidos.
- **Insertion Sort:** Ideal para vetores pequenos ou que já estão quase ordenados; também quando inserções são frequentes e incrementalidade importa.
- **Merge Sort:** Indicado para grandes volumes de dados, quando a estabilidade (manter a ordem de elementos iguais) é importante e há memória disponível para suporte.
- **Quick Sort:** Excelente escolha para dados em memória de tamanho grande, quando a ordem relativa de elementos iguais não é crítica ou pode ser controlada; cenário padrão em bibliotecas de ordenação.

6. CONCLUSÃO

Ao estudar e comparar diferentes métodos de ordenação em linguagem C, fica claro que cada algoritmo possui características próprias, podendo atender melhor determinados cenários. Mesmo que todos tenham o mesmo objetivo — organizar dados —, a forma como executam esse processo, o tempo de resposta e o uso de memória tornam alguns métodos mais adequados que outros conforme o tamanho e o tipo dos dados analisados.

Percebe-se que os métodos simples, como Bubble Sort, Selection Sort e Insertion Sort, continuam sendo importantes não só pela fácil implementação, mas também porque ajudam a entender a lógica de comparação, troca e posicionamento de valores dentro de vetores. Já os métodos mais avançados, como Merge Sort e Quick Sort, se destacam pela eficiência em grandes volumes de informação e melhor desempenho, mesmo exigindo mais recursos ou estratégias mais complexas.

Dessa forma, compreender os algoritmos e suas características permite escolher a solução mais apropriada para cada necessidade, equilibrando desempenho, simplicidade, custo de processamento e memória. Assim, o estudo desses métodos contribui tanto para o aprendizado inicial quanto para o desenvolvimento de sistemas mais eficientes e bem estruturados.

REFERÊNCIAS

SOUZA, Jackson É. G.; RICARTE, João V. G.; LIMA, Náthalee C. A. Algoritmos de Ordenação: Um Estudo Comparativo. Universidade Federal Rural do Semi-Árido (UFERSA), Curso de Ciência e Tecnologia, 2024. Disponível em: < https://periodicos.ufersa.edu.br/ecop/article/view/7082?utm_source=chatgpt.com >. Acesso em: 6 nov. 2025.

AL-KHARABSHEH, Khalid Suleiman; ALTURANI, Ibrahim Mahmoud; ALTURANI, Abdallah Mahmoud Ibrahim; ZANOON, Nabeel Imhammed. Review on Sorting Algorithms: A Comparative Study. International Journal of Computer Science and Security (IJCSS), v. 7, n. 3, p. 120–126, 2013. Disponível em: < https://www.cscjournals.org/manuscript/Journals/IJCSS/Volume7/Issue3/IJCSS-704.pdf >. Acesso em: 6 nov. 2025.

BAYMA FILHO, Paulo de Tarso de Carvalho. Análise comparativa de algoritmos de ordenação com foco na complexidade computacional e na notação Big O: Counting Sort, Merge Sort e Insertion Sort nas linguagens Java, Python e C. Caderno Pedagógico, v. 22, n. 11, p. 1–14, 2025. Disponível em: < https://ojs.studiespublicacoes.com.br/ojs/index.php/cadped/article/view/19789?utm >. Acesso em: 6 nov. 2025.

COELHO, Hebert; FÉLIX, Nádia. Métodos de Ordenação 1 — Ordenação (Selection, Insertion, Bubble, Merge). Universidade Federal de Goiás (UFG), [s.l.], [s.d.]. Disponível em: < https://ww2.inf.ufg.br/~hebert/disc/aed1/AED1_04_ordenacao1.pdf.> Acesso em: 06 nov. 2025.

VIANA, Daniel. Conheça os principais algoritmos de ordenação. TreinaWeb, São Paulo, 2020. Disponível em: < https://www.treinaweb.com.br/blog/conheca-os-principais-algoritmos-de-ordenacao/?utm_source. >. Acesso em: 06 nov. 2025.

DEVMEDIA. Algoritmos de ordenação: análise e comparação. DevMedia, 2019. Disponível em: < https://www.devmedia.com.br/algoritmos-de-ordenacao-analise-e-comparacao/28261?utm_source>.. Acesso em: 06 nov. 2025.